AMAZON S3: OBJECT STORAGE AND MANAGEMENT

TYPES OF STORAGE CLASSES IN S3

1. Amazon S3 Standard
2. Amazon S3 Glacier Deep Archive
3. Amazon S3 Standard Infrequent Access
4. Amazon S3 one-zone-IA
5. Amazon S3 Intelligent Tiering
6. Amazon S3 Glacier
    1. amazon S3 glacier instant Retrieval
    2. amazon S3 glacier flexible Retrieval

1. Amazon S3 Standard

1. S3 standard offers high durability, availability and performance object storage for frequently accessed data.
2. Provides Durability upto 99.999999999%.
3. And availability upto 99.99%
4. Supports SSL for data in transit and encryption of data at rest.
5. Getting higher storage cost but less accessible cost.
6. Provides Low latency & high performance

2. Amazon S3 IA (infrequent access)

1. S3-IA is for data that is accessed less frequently but requires rapid access when needed.
2. Charge low for storing the Objects & high for retrieving the objects
3. Provides the Durability upto 99.999999999%.
4. And availability upto 99.9% in a year.
5. Supports SSL for data in transit and encryption of data at rest.
6. Data that is deleted from S3-IA within 30 days will be charged for a full 30 days.
7. Used for backups and disaster recovery

3. Amazon S3 Intelligent Tiering

1. The S3 intelligent tiering storage class is designed to optimize cost by automatically moving data into the most cost-effective access tier.
2. It works by storing objects in two access tiers.
3. If an object in the frequent access tier is accessed it is automatically moved back to the frequent access tier.
4. There is no retrieval fees when using the S3 intelligent tiering storage class and no additional tearing fees when objects are moved between access tiers.

CLOUDBLITZ

5. Provides the low latency and high performance.
6. Objects less than 128kb cannot move to IA.
7. Provides Durability upto 99.999999999%.
8. And availability upto 99.9%.

4. Amazon S3 One-Zone IA

1. S3 one zone IA is for data that is accessed less frequently but requires rapid access when needed.
2. Data store is single AZ.
3. Ideal for those who want lower cost option of IA data.
4. It is good choice for storing secondary backup copies of on-premise data of easily re-creatable data.
5. Durability is 99.999999999%.
6. Availability is 99.5%.
7. Because S3 one zone IA stores data in a single AZ, data stored in this storage class will be lost in the event of AZ destruction.

5. Amazon S3 Glacier

S3 glacier is a secure, durable, low-cost storage class for data archiving.

TYPES OF S3 GLACIER

a) Glacier Instant Retrieval

- Archive data but millisecond access
- Good for rare access but quick retrieval needed

b) Glacier Flexible Retrieval (Old Glacier)

- Retrieval time: minutes to hours
- Cheaper than Instant

CLOUDBLITZ

6. Amazon S3 Glacier Deep Archive

1. S3 glacier deep archive is amazon S3 cheapest storage.
2. Design to retain data for long period even if for 10 years.
3. All objects stored in S3 glacier deep archive are replicated and stored across at least three geographically AZ.
4. Durability is 99.999999999%.
5. Availability is 99.9%.
6. Ideal alternative to magnetic tape libraries.
7. Retrieval time within 12 hours.
8. Storage cost is up to 75% less than for the existing S3 glacier storage class.

REPLICATION RULE

Cross-Region Replication (CRR) is a feature in S3 that automatically copies objects from one bucket to another bucket in a different AWS region.

PRACTICAL-1 CREATE REPLICATION-RULE BETWEEN REGIONS

IN MUMBAI REGION

1. create a bucket & enable versioning –> amazon-s3-bucket-mumbai

IN SINGAPORE REGION

2. create a bucket & enable versioning –> amazon-s3-bucket-singapore

IN MUMBAI REGION ---> CREATE A REPLICATION RULE

1. Click on bucket
2. Click on Management
3. Scroll to Replication rules
4. Click Create replication rule
    1. rule_name ------> crr-mumbai-to-singapore
    2. status ----------> enabled
    3. scope ----------> Choose Apply to all objects
    4. Destination ------> Choose a bucket in this account
    5. bucket name ----> amazon-s3-bucket-singapore
    6. create a new IAM role
    7. create a rule
5. upload files in amazon-s3-bucket-mumbai
6. Check amazon-s3-bucket-singapore
   {File should be automatically copied}

CLOUDBLITZ

CROSS ACCOUNT S3 REPLICATION-RULE

ACCOUNT - A (source-account)

1.create a bucket
  1.enable versioning

ACCOUNT - B (destination-account)

1.create a bucket
  1.enable versioning

2.add bucket-policy on destination-bucket (account-B)

```
{
 "Version": "2012-10-17",
 "Statement": [
  {
   "Sid": "ReplicationPermission",
   "Effect": "Allow",
   "Principal": {
    "AWS": "arn:aws:iam::<ACCOUNT-A-ID>:root"
   },
   "Action": [
    "s3:ReplicateObject",
    "s3:ReplicateDelete",
    "s3:ObjectOwnerOverrideToBucketOwner"
   ],
   "Resource": "arn:aws:s3:::destination-bucket-name/*"
  }
 ]
}
```

3.replace:

1. ACCOUNT-A-id
2. destination-bucket-name

ON ACCOUNT-A

1.create a role

1. trusted-policy---> aws service
2. use case ---------> s3
3. name ------------> s3-replication-role

2.attach permission policy to role

1. go to role----> s3-replication-role ----> add permission ---> create inlined policy

```
{
 "Version": "2012-10-17",
 "Statement": [
  {
   "Action": [
    "s3:GetReplicationConfiguration",
    "s3:ListBucket"
   ],
   "Effect": "Allow",
```

CLOUDBLITZ

```
    "Resource": "arn:aws:s3:::source-bucket-name"
   },
   {
    "Action": [
     "s3:GetObjectVersionForReplication",
     "s3:GetObjectVersionAcl"
    ],
    "Effect": "Allow",
    "Resource": "arn:aws:s3:::source-bucket-name/*"
   },
   {
    "Action": [
     "s3:ReplicateObject",
     "s3:ReplicateDelete",
     "s3:ObjectOwnerOverrideToBucketOwner"
    ],
    "Effect": "Allow",
    "Resource": "arn:aws:s3:::destination-bucket-name/*"
   }
  ]
}
```

3.Replace

1. source-bucket-name
2. destination-bucket-name

4.create a replication-rule

1. rule-name -----> cross-account-rule
2. status ---------> enabled
3. apply to ------> entire bucket
4. destination ---> destination-bucket-name
5. destination account ID
6. iam role ------> use existing iam role ---> s3-replication-role
7. ownership -----> change object ownership to destination bucker owner
8. save

5.Test

1. upload file in source-bucket
2. check in the destination-bucket

LIFECYCLE RULE

1. Amazon S3 Lifecycle Policy is a rule that automatically moving the objects in the most cost-effective storage classes.
2. It works by defining lifecycle rules that move objects between different storage classes or permanently delete them after a specified number of days.

PRACTICAL

1. create a bucket
2. click on bucket and upload a file
3. go to management and click on lifecycle_policy and add rule
    1. name--> lifecycle
    2. apply all object in bucket
    3. tag --> lifecycle-rule
    4. current version
        1. standard IA----> 30 days
        2. standard one zone----> 60 days
        3. s3 glacier-------> 180
4. previous version
    1. standard IA------> 30 days
    2. one zone --------> 60 days
    3. glacier deep ----> 120 days
5. deleted previous version----> 365 days
6. clean up delete marker
7. Then save

CLOUDBLITZ

ENCRYPTION IN S3

What is encryption

Encryption is the process of converting readable data (plaintext) into unreadable data (ciphertext) using an encryption key

Example:

Original file
|
| Encryption + Key
↓
Encrypted data

Suppose we have
1.file.txt
hello mayur

After encryption, the stored data looks like meaningless encrypted information like
1.file.txt
1 0 1 0 1 0

When an authorized user downloads it:

Encrypted data
|
| Decryption + Key
↓
Hello Mayur

Why do we need encryption

1. "Suppose someone gets access to our S3 storage. Can we protect the actual data?"
   { Yes with Encryption we can protect our data }

Example:

Without Encryption:
S3
|
└── file.txt
↓
readable data

With Encryption:
S3
|
└── file.txt
↓
encrypted data

Even if encrypted data is obtained, the attacker should not be able to understand the original data without the required decryption capability/key.

CLOUDBLITZ

Encryption in S3

S3 provides server-side encryption, where AWS encrypts objects before storing them and decrypts them when an authorized request retrieves them.

Types of Encryptions in s3

1. SSE-S3
2. SSE-KMS
3. DSSE

1.SSE-S3

Server-Side Encryption with Amazon S3 managed keys

S3
 |
 └── AWS manages encryption keys

AWS manages the encryption key for you.
You don't manage the KMS key yourself.

2.SSE-KMS

Server-Side Encryption with AWS KMS keys

S3
 |
 | encryption/decryption
 ↓
AWS KMS
 |
 └── KMS Key

CLOUDBLITZ

3.DSSE

Dual-layer server-side encryption using KMS

AWS KMS (key management service)

AWS KMS is a managed service used to create and control cryptographic keys that can be used to protect data.

Two Types of KMS Keys

1. Symmetric
2. Asymmetric

1.Symmetric

The same cryptographic key is used for encryption and decryption.

"Authentication and authorization decide who can access the object. Encryption protects the data itself."

2.Asymmetric

Asymmetric encryption uses two mathematically related keys:

1. Public key (can share with everyone)
2. Private key (only user keeps it)

Public key can be shared. Private key must be protected.

What happens when we disable the KMS key

Initially:

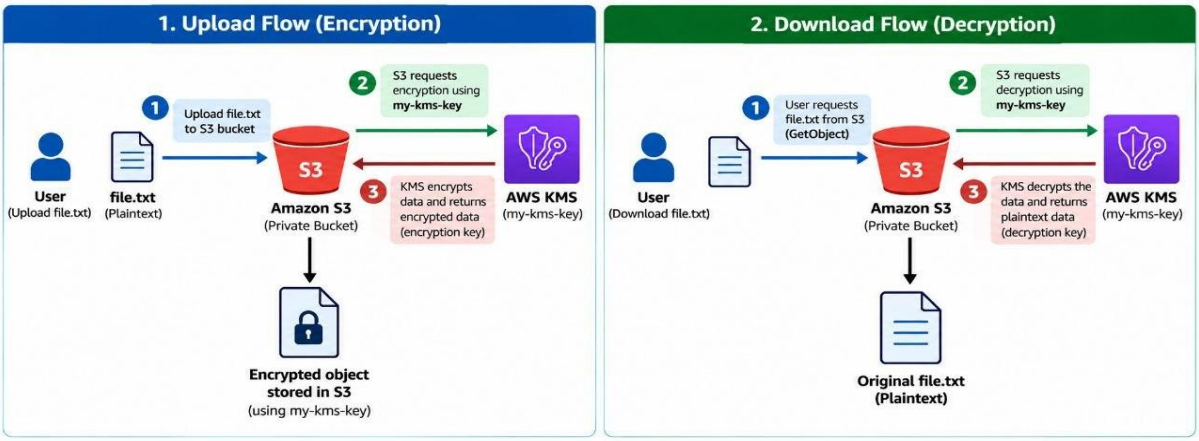
KMS Key
|
└── Enabled
|
↓
Download works

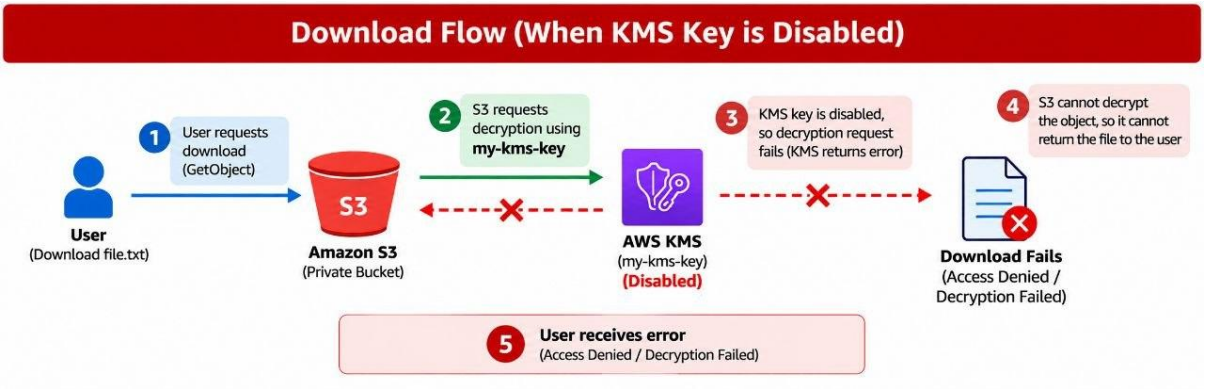
CLOUDBLITZ

When you disable:

KMS Key
|
└── Disabled
|
↓
KMS cryptographic operations
using that key
↓
unavailable

If an SSE-KMS key is disabled, operations requiring that key for decryption will fail until the key is enabled again.





HOW TO CREATE A KMS KEY & APPLY TO BUCKET

1.go to KMS (key management service)

1. click on customer manage keys ----> create_keys
2. key_type ---------> symmetric
3. key_usage -------> encrypt and decrypt
4. advanced setting
    1. Key type ---> Single-region
    2. Alias --------> my-kms-key
    3. Key administrators -------> mayur
    4. key users-------> mayur
    5. create a key

2. create a s3 bucket & bucket should be private also add encryption

1. open bucket
2. Go to properties
3. default encryption
4. select ------> SSE-KMS
5. choose key ----> my-kms-key
6. save changes

3. Upload a file to s3 bucket

1. upload file.txt
2. open file -----> file properties
3. check encryption ----> AWS KMS ---> my-kms-key

8.Test Download (Before disable)

1. download file.txt
   {successfully download file.txt}

9.Disable KMS key

1. go to KMS
2. click on my-kms-key ----> action -----> disable

10.Test download again

1. download file.txt
   {access denied/Decryption failed}

Note:

1. KMS is used for encryption + decryption
2. If key is disabled → data becomes unreadable
3. Default encryption applies only to new files
4. S3 public access does NOT bypass KMS
5. Without an active KMS key, encrypted S3 objects cannot be accessed.

ADDING ENCRYPTION TO THE BUCKETS

CLOUDBLITZ

1. create a bucket
2. upload the objects
3. go to the bucket policy ----> policy generator
    1. s3 bucket policy
    2. *--> for all the object
    3. put object/put bucket policy
    4. paste the bucket ARN
    5. string not equals
    6. key--> server-side encryption
    7. aws:kms
    8. add statement
    9. generate policy

DIFFERENCE BETWEEN THE KMS AND BUCKET ENCYRPTION

AWS Key Management Service (KMS)

1. KMS creates and manages encryption keys.
2. It is used to encrypt and decrypt data.
3. You can control who can use the key.
4. If the key is disabled, data cannot be read.

Bucket Encryption / Policy

1. It forces users to upload encrypted files.
2. It checks if encryption (like KMS) is used or not.
3. If encryption is not used, upload is denied.
4. It does not create keys, only enforces rules.
